

Clases genéricas y Colecciones

Clases genéricas

Ejercicio 1 – clase Par

En ocasiones, necesitamos que un mismo método devuelva dos valores de distinto tipo, **por ejemplo un String y un valor entero** ¿sería esto posible?

Efectivamente, la solución es devolver una objeto de una clase que permita almacenar esos dos valores como atributos. Implemente la **clase Par** que permita almacenar dos datos: clave de tipos String y valor de tipo entero. Añade los métodos necesarios para crear y consultar esos dos valores y prueba su funcionamiento.

```
{Granada,18}  
Clave: Granada  
Valor: 18
```

Ejercicio 2 – clase ArrayInt

Vas a crear tu primera colección: implementa la clase **ArrayInt** que permita almacenar en un array varios valores de tipo entero. El tamaño del array se pasa como argumento al constructor de la clase. La clase dispone de métodos para guardar/consultar los valores a través de su posición.

```
[25, 0, 0, 18, 0]  
Valor 0: 25
```

Ejercicio 3 – clase ParGenerica

A partir de la clase **Par**, implementa una clase **ParGenerica** que almacene dos datos de cualquier tipo. Esta clase debe permitir consultar los datos de los atributos, pero no puede permitir que sean modificados después de haber sido creados. Prueba la funcionalidad de esta clase en un programa Java.

Ejercicio 4 – clase ArrayGenerico

A partir de la clase **ArrayInt**, crea:

1. Una clase nueva **ArrayGenerico** que permita almacenar cualquier tipo de objetos.
2. Añade un método que permita buscar la posición de un valor. Si ese valor no se encuentra en el array, devuelve -1 .
3. Otro método que permita al usuario añadir un elemento al array. El método busca un hueco para añadir el elemento. Si el array está lleno, lanza una excepción.
4. Prueba su funcionamiento en un programa:
 - a) Crea un ArrayGenerico de 10 Integer.

- b) Guarda el valor 50 en la posición 0 y el valor 30 en la posición 1.
- c) Muestra el contenido del ArrayGenerico.
- d) Busca el valor 20 (Debe de indicar que no está).
- e) Busca el valor 30 (Debe indicar que está en la posición 1).
- f) Añade el valor 100 en el primer hueco libre.
- g) Muestra todos los valores por pantalla con el siguiente formato

```
[50, 30, null, null, null, null, null, null, null, null]
Busco el 20:-1
Busco el 30:1
**** Valores ****
50
30
100
null
null
null
null
null
null
null
```

5. Contesta:

- a) ¿Has encontrado algún problema al manejar clases genéricas?
- b) Prueba a llenar el ArrayGenerico y a añadir un nuevo valor. ¿Qué ocurre?
- c) ¿Existe alguna forma de añadir un valor si el array está lleno?

Colecciones

Ejercicio 5 – Control de asistencia

Imagina que estás desarrollando un sistema de control de asistencia a clase. Implementa un programa usando una colección **HashSet** que contenga nombres de alumnos que hayan asistido a clase (pueden ser ficticios), y muéstralos por pantalla haciendo uso de la sintaxis de **foreach**. Prueba a añadir un duplicado y observa lo que ocurre.

Ejercicio 6 – Agenda

Estás desarrollando una aplicación para gestionar contactos, y debes asegurarte de que no existan contactos duplicados y que estos aparecen en orden alfabético. Crea un programa en Java que haga uso de un **TreeSet** para almacenar objetos de la clase **Contacto**, que deberéis implementar previamente y que contenga, como atributos de tipo String, el nombre, teléfono y correo del contacto. Muestra los contactos por pantalla y prueba a añadir un duplicado.

Ayuda: los **TreeSet** son conjuntos ordenados por lo que hay que decirles como se ordenan nuestras clases.

Ejercicio 7 - El Podio del Torneo (Ideal para ArrayList)

En este ejercicio vas a crear un sistema que gestiona las puntuaciones y los nombres de los jugadores en un **torneo de eSports**. Como los espectadores quieren ver constantemente quién va en primera, segunda o tercera posición, un **ArrayList** es perfecto porque acceder a los índices es instantáneo.

Crea un programa con una clase principal que haga lo siguiente paso a paso:

- Instancia un **ArrayList** de tipo String llamado **clasificacion**.
- Añade, en este orden, a los siguientes jugadores al final de la lista: "Ana", "Carlos", "Beatriz", "David", "Elena".
- El imprevisto:** hubo un error de arbitraje. Resulta que "Beatriz" hizo trampa. Elimínala de la lista usando su nombre (no su índice).
- La remontada:** "David" acaba de ganar una partida espectacular y sube a la primera posición. Mueve a "David" al índice 0 sin borrar a "Ana" (ella y el resto deben desplazarse hacia abajo).
- El reemplazo:** "Carlos" se retira del torneo por problemas técnicos y es sustituido por "Zacarías". Reemplaza el valor en la posición de Carlos por "Zacarías" usando el método **.set()**.
- Imprime por consola el podio (los 3 primeros jugadores) usando un bucle for clásico y el método **.get(i)**.
- ¿Cambiaría el problema si usamos **LinkedList** en lugar de **ArrayList**? ¿Cuál sería el cambio?

Ejercicio 8 - El Historial del Navegador (Ideal para LinkedList)

Vas a simular cómo funciona el botón de "Atrás" y el historial de pestañas de un navegador web como Chrome o Firefox. Para esto, una **LinkedList** es la estructura de datos perfecta, ya que cuenta con métodos exclusivos para manipular el principio y el final de la lista al instante.

Crea un programa que simule la navegación web siguiendo estos pasos:

- Instancia una **LinkedList** de tipo String llamada **historial**.
- El usuario visita, en orden, estas páginas. Como la última página visitada es siempre la más reciente, añádelas usando el método exclusivo **.addFirst()**:
 - ["https://www.google.com/search?q=google.com"](https://www.google.com/search?q=google.com)
 - "wikipedia.org/Java"
 - "stackoverflow.com"
 - "github.com"
- Imprime cuál es la página actual (la más reciente) usando **.peekFirst()** sin borrarla de la lista.
- El límite de memoria:** Tu navegador está configurado para recordar solo las 3 últimas páginas. Como actualmente tienes 4, elimina la más antigua (la que está al final de la lista) usando el método exclusivo **.removeLast()**.
- Botón Atrás:** El usuario pulsa el botón de retroceder. Elimina la página actual (la primera) usando **.removeFirst()** y guárdala en una variable temporal. Luego, imprime un mensaje que diga: "Saliendo de [página eliminada], volviendo a [nueva página actual]".
- Imprime el estado final del historial completo.
- ¿Cambiaría el problema si usamos **ArrayList** en lugar de **LinkedList**? Coméntalo.

Ejercicio 9 – Lista de clase

Recupera la clase **Alumno** que implementaste en la unidad anterior. Si no la tienes implementada, crea una clase **Alumno** que almacene su nombre y su edad, e implementa los *getters* para ambos atributos, así como su *constructor*.

Implementa una clase llamada **ListaAlumnos**, que implemente los siguientes métodos:

- void agregarAlumno(Alumno alumno):** Añade un alumno a la lista.
- void mostrarAlumnos():** Muestra todos los alumnos en la lista.
- Alumno buscarAlumno(String nombre):** Busca la primera ocurrencia de un alumno por su nombre y lo devuelve. Si no lo encuentra, devuelve null.

Implementa un programa que:

- Cree una lista de estudiantes
- Cree 2 alumnos y los añada a la lista
- Muestre la lista
- Busque un alumno por nombre y muestre la posición en la lista.
- Modifica un alumno y vuelve a mostrar la lista ¿Qué ocurre?

Recuerda que para este ejercicio deberás hacer uso de **ArrayList** o **LinkedList**.

Ejercicio 10 – Diccionario castellano/inglés

Crea un **diccionario** de palabras donde tengamos como llave una palabra en castellano y su valor sea una palabra en inglés. Implementa directamente en un programa un objeto de **HashMap** que nos permita almacenar esa información, y haz los siguientes pasos:

- Añade cinco palabras y sus respectivas traducciones.
- Busca la traducción de una palabra en castellano y obtén su traducción al inglés.
- Elimina una palabra del diccionario
- Muestra todas las palabras, junto con sus traducciones recorriendo el diccionario.

Ejercicio 11 - Provincias

- Implementa una clase **Provincia** para almacenar las ocho provincias de Andalucía: abreviatura, nombre y código postal de cada provincia. Incluye los métodos **gets** y **toString**.

```
Provincia p1 = new Provincia("GR","Granada",18);
Provincia p2 = new Provincia("J","Jaen",23);
Provincia p3 = new Provincia("AL","Almeria",04);
Provincia p4 = new Provincia("CO","Cordoba",14);
Provincia p5 = new Provincia("SE","Sevilla",41);
Provincia p6 = new Provincia("MA","Malaga",29);
Provincia p7 = new Provincia("CA","Cadiz",11);
Provincia p8 = new Provincia("H","Huelva",21);
```

- En un programa, vamos a crear 3 colecciones utilizando cada uno de los tipos de colecciones Map que has visto: **HashMap**, **LinkedHashMap** y **TreeMap**. Cada una de ellas almacenará las ocho provincias guardando como clave el código postal (<Integer>,<Provincia>). Muestra el resultado por pantalla. ¿Qué diferencias observas entre los 3 tipos de Map?
- Duplica la provincia de Granada en los tres casos ¿Qué ocurre y por qué? ¿Necesitaríamos sobrescribir **equals**, **hashCode** y **compareTo**?

Ejercicio 12 – Provincias versión 2

Modifica el **ejercicio 11** de manera que los **3 Map** almacenen sus datos guardando como clave la instancia de **Provincia** y como valor su código postal (`<Provincia><Integer>`).

- ¿Hay algún problema con el **HashMap**? Compáralo con el **ejercicio11**.
- ¿Hay algún problema con el **LinkedHashMap**? Compáralo con el **ejercicio11**.
- ¿Hay algún problema con el **TreeMap**? Si es así, plantea una solución. Ayuda: los **TreeMap** son **Map** ordenados, hay que decirles como se ordenan nuestras clases.
- Al igual que en el ejercicio 3, duplica la provincia de Granada en los tres casos ¿Qué ocurre y por qué? ¿Necesitaríamos sobrescribir **equals** y **hashCode**?

Ejercicio 13 - Inventario

Imagina que estás implementando un programa para la gestión de inventario de tiendas, y para ello te piden que implementes una colección de datos llamada **Inventario** para almacenar los productos, de los cuales nos interesa conocer su nombre, marca, precio y stock (cantidad del producto que tenemos en almacén).

Tenemos dos tipos de productos: **Ordenadores**, de los que queremos conocer los nombres de algunos de sus componentes (CPU, RAM, Placa base y almacenamiento); **Teléfonos**, de los que queremos tener en memoria otros componentes distintos, como tamaño de batería, tipo de señal (5G, 4G, etc.) que admite, y su autonomía.

Debes implementar la siguiente funcionalidad para el inventario:

- `void anadirProducto(producto, cantidad)`**: añade un producto al inventario.
- `void actualizaStockProducto(producto, nueva_cantidad)`**: actualiza el stock del producto en el inventario.
- `void eliminarProducto(producto)`**: Elimina el producto del inventario.
- `void mostrarInventario()`**: muestra por pantalla los productos que haya en el inventario.

Considera si debieras implementar más clases además de **Inventario** y, de hacerlo, determina qué atributos deben tener getters o setters. Razona por qué los has implementado (o por qué no los has implementado). Además, deberás implementar un programa que compruebe el correcto funcionamiento de dichos métodos, además de los métodos auxiliares y constructores que estimes oportunos.

Ejercicio 14 - Zoológico

Desarrolla un sistema de gestión para un zoológico que permita agregar, eliminar y listar **animales**, de los cuales queremos conocer su nombre, edad y hábitat. Los hay de dos tipos: **mamíferos**, de los que necesitamos saber si son domésticos o no; y **aves**, de los que queremos conocer la envergadura de sus alas.

Implementa una clase llamada **Zoológico** que almacene los animales descritos e implemente la siguiente funcionalidad:

- **void agregarAnimal(Animal animal)**: Agrega un animal al zoológico.
- **void eliminarAnimal(String nombre)**: Elimina un animal del zoológico por su nombre.
- **void listarAnimales()**: Lista todos los animales del zoológico. (Pista: debes hacer uso de un iterador).

Además, debes implementar un programa principal donde se prueben toda la funcionalidad que hayas implementado en este ejercicio.

Ejercicio 15 – El carrito de la compra

Vas a gestionar un caso típico de este ciclo, la limpieza del carrito de la compra de un **sitio web de e-commerce**. De cada artículo interesa conocer: id, nombre, precio y disponibilidad.

Imagina que un usuario tiene varios artículos en su carrito de una plataforma tipo Amazon o eBay. Antes de proceder al pago, el sistema debe realizar una **validación final** siguiendo las siguientes reglas del negocio. Se deben eliminar del carrito:

1. Aquellos artículos que **ya no tengan stock disponible** (disponibilidad = 0).
2. **Los duplicados erróneos**: productos con precio 0€ o negativo por un error del sistema.
3. **Cupones caducados**: representados como items con un nombre que empieza por "EXP-".

Implementa un programa que gestione un carrito de la compra en el que se puedan ir guardando artículos según el criterio del usuario. Para ello:

1. Usa una colección (tu decides cual sería la ideal).
2. Añade los siguientes productos:

("P001", "Teclado Mecánico", 45.50, 12)

("P002", "Ratón Gaming", 15.0, 5)

("P003", "EXP-DESCUENTO10", -10.0, 1)

("P004", "Monitor 4K", 350.00, 0)

("P005", "Alfombrilla XL", 0.0, 20)

Como puedes observar, hay un cupón caducado, un producto con un precio incorrecto y un producto sin stock.

3. Muestra el carrito antes de validar
4. Valida el carrito, muestra los productos tras aplicar las tres condiciones y el pvp total.

Ejercicio 16 - El super carrito de la compra

Modifica el ejercicio anterior de manera que:

1. Si se encuentra un producto en oferta, por ejemplo "Ratón Gaming" se añade automáticamente una "Funda de Regalo" justo después.
2. Si encuentra un producto agotado, lo reemplaza por una "Tarjeta Regalo".
3. Muestra el resumen final empezando por el último producto añadido.
4. ¿Serías capaz de mostrar los productos ordenados por precio?

Ordenación y búsqueda

Ejercicio 17 – Ordenando el array

A partir de la clase **ArrayInt** que ya has utilizado en otros ejercicios, crea un programa que guarde 50 números aleatorios en el intervalo [0-100].

- a) Muestra el array generado por pantalla.
- b) Ordena el array y vuelve a mostrarlo.

Ejercicio 18 – Seguimos ordenando

A partir del ejercicio 9, ordena la lista de alumnos antes de mostrarla por pantalla. En este caso, no se trata de elementos con un orden natural por lo que tendrás que especificar el orden de ordenación.

Ejercicio 19 – Búsqueda lineal vs búsqueda binaria

A partir del ejercicio 17, vamos a realizar un cálculo de eficiencia comparando el método **buscarElemento** ya implementado frente al de búsqueda binaria **Array.binarySearch**. Para ello, tendrás que crear los métodos correspondientes para ordenar y realizar una búsqueda binaria.

Para el calculo de tiempos usa **System.nanoTime()** para marcar los inicios y finales de los cálculos.

¿Qué método es más eficiente?